Project Phase #3

Q1 Delivered Increment

(Team work - complete Sections 1-5 and 7-9 within your team)

Q1.1 Increment description

(4-6 sentences + one entry per requirement): Describe the coherent increment your team delivered. Name the user-visible capabilities, the users who benefit, and the regulatory obligation it supports. Then add one entry for every requirement you worked on in this phase, with its delivery status and the acceptance criteria that demonstrate it.

Optional: if your system models capability/requirement status internally (e.g., in a database table), you're welcome to generate this section directly from that data instead of writing it by hand. Just make sure each entry still follows the structure below.

- **Delivered capability:** [what a user can now do through the common interface]
- **Users/stakeholders:** [who benefits]
- **Regulatory provision(s):** [citation]
- **REQ-.. - [short name]** │ Status: [complete / partial] │ Demonstrated by: [AC-..]
    - *If partial:* [what is still missing]

Q1.2 Comparison with the Phase # 2 plan

(3-4 sentences): Compare what you delivered with the Phase 3 milestone in Phase 2, Section 6.1. State which planned items were delivered, and which were not. Instructor-required scope changes are rare — note them with their date if they occurred. A pod may not unilaterally reduce shared scope because one team fell behind; if your pod believes a shared requirement needs to change, both teams must agree and get instructor sign-off before recording it here.

Q2 Deployment and Operation

(Team work - complete within your team)

Q2.1 Deployed web service

(3-4 sentences): Explain where and how the service is deployed (hosting platform, region, runtime), how the deployed version maps to a tag or commit in your repository, and how a new version is released. State whether the service is always on. If it is not, explain why and when it

is unavailable. Note any costs, rate limits or usage restrictions podmates and graders must respect.

Q2.2 Registry and team documentation

Confirm that the class registry of deployed services and your team's project documentation were updated in this phase. The registry entry must identify the deployed web service and the automated test runner. The team documentation must describe the system as delivered, including relevant models, interfaces, functionality and known limitations.

- **Registry entry:** [location] │ **Updated on:** [date]
- **Team documentation:** [location] │ **Updated on:** [date]

Q2.3 Operational experience supported by Observability evidence

(3-5 sentences): Report how your Phase 2 plan (Section 7.3) played out in practice. Explain how persistent data is stored and reset, how secrets and credentials are managed, and which logs or metrics you use to detect failures. Include one short excerpt (log lines, dashboard screenshot or metric) showing that these signals exist in the deployed system.

Q3 Implementation and Interface Conformance

(Team work - complete within your team)

Q3.1 Operation status

(One entry per operation + 2-3 sentences): List every operation in the pod's common interface contract (Phase 2, Section 5.1), not only the ones you implemented. For each, state whether your deployed service implements it and whether its success and error responses conform to the contract. Then explain, in 2-3 sentences, how the new functionality integrates with the existing system: which components changed and how requests flow through them.

Tip: you will be asked to report this table again in the next phases. Consider having your test suite emit it directly (e.g., a script that runs your contract tests and prints the method/path, implemented status, and conformance) instead of filling it in by hand each time.

- **[METHOD /path]** │ **Requirement(s):** [REQ-..] │ **Implemented:** [yes / partial / no]
    - **Success responses conform:** [yes / no + difference]
    - **Error responses conform:** [yes / no + difference]
- **Integration:** [components changed and how requests flow through them]

Q3.2 Contract changes and pod communication

(2-4 sentences): State whether your pod's common interface contract (Phase 2, Section 5.1) changed during this phase. For each change, give the new version, what changed, why, and when and how the pod agreed to it. If your implementation intentionally deviates from the contract, identify the deviation and the plan to resolve it. If nothing changed, state that explicitly.

- **Contract version at delivery:** [version]
- **Changes since Phase 2:** [change and reason, or none]
- **Pod agreement:** [date and channel]
- **Intentional deviations and resolution plan:** [deviation/plan, or none]

Q4 Automated Validation

(Team work - complete within your team)

Q4.1 Test runner and execution instructions

Since you've been given an OpenAPI spec for the common interface contract, you do not need to write instructions for building or running your own test suite. Instead, confirm that your deployed service is reachable and conforms to that spec.

- **Base URL override:** [how a grader points the suite at an arbitrary team's base URL, without editing test code]
- **Response received:** [status + body]
- **Conforms to spec:** [yes/no]
- **Known deviations from the OpenAPI spec:** [list them, or state "none"]

Q4.2 Tests for new functionality

(One entry per AC + 2-3 sentences): Map each acceptance criterion addressed in this phase to the automated tests that check it. Include at least one failure-path test for each core operation delivered. Then explain, in 2-3 sentences, how the expected outputs (test oracles) were established and why you trust them.

- **AC-..** │ **Requirement(s):** [REQ-..]
    - **Test(s):** [test names or IDs] │ **Type:** [success / failure / boundary] │ **Automated:** [yes / no + reason]
    - **Oracles:** [how expected outputs were established and why they are trustworthy]

Q4.3 Regression and static checks

(3-4 sentences): Explain how you prevent regressions in functionality that already existed (e.g., re-running prior automated tests in CI, contract tests across phases).

- **CI run:** [link]

- [Your explanation here]

Q4.4 Static checks

(2-3 sentences): Identify the static checks you run (linting, type checking, security or dependency scanning, contract validation), when they run (pre-commit, CI on pull request, before deployment), and whether a failing check blocks merge or deployment. Link to a recent CI run. If these haven't changed since the last phase, state that explicitly instead of repeating the same answer.

- **CI run:** [link]
- [Your explanation here]

Q4.5 Test results against the deployed system

Report the results of running your full suite against your own deployed service, at the version on the cover page. Explain each failing test: its cause, whether it is a defect in the system, the test or the specification, and the plan to resolve it.

- **Run date/time:** […] │ **Version tested:** [tag/SHA]
- **Tests run:** [#] │ **Passed:** [#] │ **Failed:** [#] │ **Skipped:** [#]
- **Failure:** [test] │ **Classification:** [system / test / specification] │ **Cause:** […] │ **Plan:** [fix and target phase]

Q5 Updated System Models

(Team work - complete within your team)

Q5.1 Model updates

Update the models from Phase 2 (Sections 3.2 and 7.1) to reflect the system as it was actually delivered, rather than as it was originally planned. For each model, record whether it changed and why, and insert the current version. Because LLM-assisted drawing is permitted, models are held to a high standard: consistent notation, labeled elements and relationships, and no components that do not exist in the implementation.

- **[Model name and type]** │ **Changed:** [yes / no] │ **What changed and why:** […] │ **Location:** [file/page]
- *(Insert updated diagram(s) here, or identify the attached artifact and the pages/sections to review)*

Q5.2 Model-implementation consistency

(3-4 sentences): Explain how you checked that the models match the implementation (e.g., generated from code, reviewed against the contract, walked through by a teammate). Identify at least one place where the implementation diverged from a Phase 2 model and state which one you corrected: the model or the code.

Q6 Pod Cross-Team Readiness

(Pod work - coordinate with the other team in your pod; each team writes its own section)

Q6.1 Cross-team test

(2-3 sentences + one entry per direction): Phase 3 makes your service available for cross-team testing. Before submitting, run your full test suite at least once against each podmate's service, and have at least one podmate run their full suite against yours. In Phase 3, the goal is only to confirm that the common interface works and that you get back well-formed reports. You are not expected to have fixed every mismatch yet. Starting Phase 4, this will be factored into grading. Record the results, what happened when something did not work, and report coverage (code, component, and system-model coverage) for each direction.

- **Our tests → [team] service** │ **Date:** […] │ **Run/passed:** [#/#] │ **Main problem and owner:** […, or none]
- **[team] tests → our service** │ **Date:** […] │ **Run/passed:** [#/#] │ **Main problem and owner:** […, or none]

Q6.2 Specification issues discovered

(3-4 sentences): Implementation and cross-testing may expose ambiguities in the shared specification. Identify at least one requirement, acceptance criterion or interface detail that teams interpreted differently or found underspecified. Explain its effect and the pod's plan to resolve it. Keep the common specification up to date.

- **Issue:** [REQ/AC/operation affected and the ambiguity]
- **Effect:** [how implementations or tests diverged]
- **Pod decision or plan:** [decision and date, or plan and deadline]

Q7 Status, Limitations and next phase Adjustments

(Team work - complete within your team)

Q7.1 What changed and what remains incomplete

(3-5 bullets each): Summarize what changed in the system since Phase 2, then list what remains incomplete from the pod's planned scope. Use REQ and AC identifiers.

- **Changed since Phase 2:** [changes, with REQ/AC identifiers]
- **Remaining incomplete:** [scope items, with REQ/AC identifiers]

Q7.2 Known limitations and defects

List known limitations and defects of the deployed system that podmates or users could encounter: unsupported inputs, performance limits, missing error handling, security gaps or failing tests. A limitation you disclose here is less costly than one a podmate discovers.

- **LIM-01** │ **[limitation or defect]** │ **Affects:** [REQ/operation] │ **Severity:** [high / medium / low]
    - **Workaround for podmates:** […, or none] │ **Target phase:** [4 / 5 / 6]

Q7.3 Adjustments to next phase milestone

Based on what you learned, state whether the Phase 4 milestone (Phase 2, Section 6.2) still holds. Describe any change to its capability, sequencing or risks, and whether it requires pod agreement. Update your priority risks from Phase 2, Section 9.1: which were reduced, which remain, and any new risk.

Q8 GenAI Use in This Phase

(Team work - complete within your team)

Q8.1 Actual GenAI use

Compare actual GenAI use in this phase with your Phase 2 plan (Section 8.2). Answer each item below **(3-4 sentences per teammate)**:

- **Tools and delegation:** State the GenAI tools used and the work delegated (code, tests, models, documentation).
- **Verification:** Explain how outputs were verified before being merged or submitted.
- **Correction example:** Give one concrete example where GenAI output was wrong or needed correction, and how you caught it.
- **Agentic principles:** Discuss the extent to which you adhered to the principles of agentic software engineering described in the course, or deviated (honesty please!).

Q9 References

Cite sources as appropriate; we expect this would be 5–8 distinct, relevant sources for typical teams. Number sources in the order they first appear in your report, and reuse the same number whenever you cite that source again. Technical documentation for the frameworks, platforms and services you actually used is expected in this phase.

Place citations immediately after the supported claim and identify a section or provision when needed:

- **Regulatory claim:** “The obligation includes documenting product components [1, Annex I, Part II(1)].”
- **Platform behavior:** “The hosting service restarts idle containers after 15 minutes [3, Section 4.2].”

Format each reference as follows: [Number] Author or issuing organization. “Title.” Publication date or version, if available. URL. Accessed: YYYY-MM-DD.

Example: [1] European Parliament and Council of the European Union. “Regulation (EU) 2024/2847 (Cyber Resilience Act).” October 23, 2024. https://eur-lex.europa.eu/eli/reg/2024/2847/oj. Accessed: 2026-09-08.